

WHAT THIS SESSION IS

The first real Spring AI class. It answers "why a framework at all", sets up a Spring Boot project that talks to OpenAI, then builds up three capabilities step by step: a simple chat endpoint, remembering the conversation with advisors, and letting the model call your own tools. This is a foundation session, so aim for the big picture, not memorising method names (they change often, as you will see).

CONTENTS

1. Why use Spring AI instead of raw SDK code
2. What Spring AI is and what it can do
3. Agents, harnessing, and tools (the bigger picture)
4. Setting up the Spring Boot project
5. ChatModel versus ChatClient
6. Your first AI chat endpoint
7. Adding memory with advisors
8. Reading the full response and metadata
9. The knowledge cutoff problem
10. Tool calling (function calling)
11. Key takeaways

1 Why use Spring AI instead of raw SDK code

In Session 02 you called an LLM directly from Java. That works, but it has two real problems:

- **Provider lock in.** Each LLM (OpenAI, Anthropic, Gemini) has its own SDK and its own code. If you later move from OpenAI to Anthropic, you would have to rewrite a lot. A framework gives you an **abstraction**, so switching providers is quick.
- **Complex work is error prone by hand.** Real AI work is not just chatting. You deal with embeddings, vector databases, RAG, and tool calling. Wiring all that yourself invites bugs. A tested framework lets you write only the code you care about and trust the rest.

WHERE EACH TOOL FITS

Spring AI is the framework for adding AI to Spring Boot apps. For building **agents**, this course uses two more technologies: **Google ADK** and **LangChain4j**. Spring AI does not have full agent support yet (only a workaround), but it is the natural first step because most industry projects already run on Spring Boot.

2 What Spring AI is and what it can do

Spring AI is part of the Spring ecosystem. Its aim is to streamline building applications that use AI, without unnecessary complexity. It is inspired by LangChain and other established libraries.

With just a dependency added to a Maven or Gradle project, Spring AI lets you:

- Build chat applications
- Generate images, and describe or analyse an image you provide
- Do text to speech and speech to text
- Work multimodally (text, image, audio together)
- Use embeddings, vector stores, and RAG (covered in later sessions)

VERSIONS

There are two live versions. **Spring AI 1.0** is the currently stable one in industry and comes by default with **Spring Boot 3.5**. **Spring AI 2.0** is newer and comes by default with **Spring Boot 4**. You do not pick the Spring AI version directly; it follows the Spring Boot version you choose. This course covers both, starting with 1.0.

3 Agents, harnessing, and tools (the bigger picture)

A helpful mental model from the class: **an LLM is just a brain**. It has an ear and a mouth (it can read your text and talk back), but it has no hands or legs. On its own it can only chat.

To make an LLM actually *do* things, you give it **tools**. Example: instead of manually going to Amazon, adding a MacBook to the cart, and paying, you could tell an assistant "buy a new MacBook and deliver it to my home". The search, add to cart, and payment steps in between are tools. An LLM cannot do those by itself, and building those tools is exactly the job the industry expects from you (nobody expects you to build a new model).

NOT EVERY PROBLEM IS A CHAT PROBLEM

Some problems are solved by talking, others by taking actions. Actions need tools. Tool calling (Section 10) is your first taste of giving the brain some hands.

4 Setting up the Spring Boot project

1. Create a new **Spring Boot** project (Maven) from Spring Initializr.
2. Add the **Spring Web** dependency (you are building REST endpoints) and the **OpenAI** Spring AI dependency (scroll to the AI section in the dependency list).
3. Java 17 is the minimum for Spring Boot. Java 21 or 25 are fine too, thanks to backward compatibility.

When you run the empty app, you get an error straight away:

```
OpenAI API key must be set. Use the connection property: spring.ai.openai.api-key
```

Create an API key on the OpenAI API platform and add it to

`src/main/resources/application.properties` :

```
spring.ai.openai.api-key=YOUR_OPENAI_API_KEY
```

NOTE ON PREDEFINED KEYS

`spring.ai.openai.api-key` is a predefined property key, just like `spring.datasource.url`. A property key can only be used once, the same way a Java variable name cannot hold two values. Anything you put in `application.properties` can also be hard coded in a Java config class, but the property file is the clean way.

5 ChatModel versus ChatClient

Spring AI gives you two levels of abstraction to talk to a model. Both are abstractions; you write no HTTP code yourself.

	ChatModel (for example OpenAiChatModel)	ChatClient
Level	Lower level	Higher level
Tied to a provider?	Yes, the class name says OpenAI	No, it is generic across providers
How you get it	Injected by Spring (constructor injection)	You build it from a model or a builder
Simplest call	<code>model.call(prompt)</code> returns a String	<code>chatClient.prompt(text).call().content()</code>

Because `ChatClient` is provider agnostic, which actual model it uses is decided by the **dependency and API key** present in your project, not by a favourite it picks on its own. It is not a human, so there is no default or favouritism.

6 Your first AI chat endpoint

Option A: using ChatModel

```
@RestController
public class TeluskoController {

    private final OpenAiChatModel model;

    public TeluskoController(OpenAiChatModel model) { // injected by Spring
        this.model = model;
    }

    @GetMapping("/info/{prompt}")
    public String getInfo(@PathVariable String prompt) {
        return model.call(prompt); // simplest: String in, String out
    }
}
```

Hit `/info/tell me about Spring AI` in Postman and the answer comes straight from the AI, with no hard coding and no database.

Option B: using ChatClient (provider agnostic)

```
private final ChatClient chatClient;

// create from a specific model...
public TeluskoController(OpenAiChatModel model) {
    this.chatClient = ChatClient.create(model);
    // ...or with the builder:
    // this.chatClient = ChatClient.builder(model).build();
}

@GetMapping("/info/{prompt}")
public String getInfo(@PathVariable String prompt) {
    return chatClient.prompt(prompt) // request specification
        .call() // call response specification
        .content(); // the String content
}
```

WHY THE METHOD CHAINING

`ChatClient` does not have a direct `call()` that returns a `String`. Instead `prompt(...)` returns a request spec, `call()` returns a response spec, and `content()` pulls out the text. Chaining them keeps the code short. You can create multiple `ChatClients` (one per provider) when you need more than one model in the same app.

7 Adding memory with advisors

By default the model has no memory of your conversation. Ask a joke, then say "more", and it replies "your message is incomplete". Each request is independent. To fix this, Spring AI uses **advisors**.

WHAT AN ADVISOR IS

An advisor adds extra behaviour to a `ChatClient`. Think of it as "do not just create a chat client, create one that is advised with something". Chat memory is *one* use case of advisors; there are others you will meet later.

To give the chat memory, attach a `MessageChatMemoryAdvisor` backed by a `ChatMemory`:

```
// a memory store
ChatMemory chatMemory = MessageWindowChatMemory.builder().build();

this.chatClient = ChatClient.builder(model)
    .defaultAdvisors(
        MessageChatMemoryAdvisor.builder(chatMemory).build()
    )
    .build();
```

In newer Spring AI (1.0.8 in the class), each conversation needs a **conversation id**. Older versions generated it automatically; the newer one asks you to pass it. Generate one and supply it on each call:

```
String conversationId = UUID.randomUUID().toString();

String answer = chatClient.prompt(prompt)
    .advisors(a -> a.param(ChatMemory.CONVERSATION_ID, conversationId))
    .call()
    .content();
```

Now "tell me a joke" followed by "more" continues the same thread, so the app becomes **stateful**.

A REAL LESSON FROM THE LIVE CLASS

Code that worked two weeks earlier broke here because a small version update (1.0.8) started requiring the conversation id. The AI ecosystem changes fast. This is exactly why you should understand concepts, not memorise exact method or bean names. No interviewer expects you to recite method signatures either.

8 Reading the full response and metadata

Instead of `.content()` (just the text), you can ask for the whole `ChatResponse` to read metadata such as which model actually answered:

```
ChatResponse response = chatClient.prompt(prompt).call().chatResponse();

// which model replied, for example gpt-4o-mini
var model = response.getMetadata().getModel();

// just the text
String text = response.getResult().getOutput().getText();
```

WHICH MODEL RUNS?

In the demo the response reported `gpt-4o-mini`. Spring AI does not choose the model for you; it depends on your API key and plan. You can explicitly set and change the model, which is covered in a later session.

9 The knowledge cutoff problem

Ask the model "when were you last trained?" and it answers with a cutoff date (for example October 2023). Ask "what is the current date and time?" and it cannot answer. Ask "did RCB win the IPL in 2026?" and it says it has no information, because that event is after its training.

TWO WAYS TO FIX THIS

An LLM only knows what it was trained on. To give it fresh or private knowledge, there are two patterns: **RAG** (Retrieval Augmented Generation, covered in later sessions) and **Tool calling** (below). Interestingly, the public ChatGPT can answer recent questions because it already uses these techniques behind the scenes.

10 Tool calling (function calling)

Tool calling lets the model interact with your own code and external APIs to get things it cannot know by itself. You define a tool as a normal Spring bean method annotated with `@Tool` .

Step 1: define a tool

```
@Component
public class DateTimeTool {

    @Tool(description = "Get the current date and time")
    public String getCurrentDateTime() {
        return LocalDateTime.now().toString();
    }
}
```


Step 2: offer the tool to the model

```
@Autowired
private DateTimeTool dateTimeTool;

String answer = chatClient.prompt(prompt)
    .tools(dateTimeTool)    // just make it available
    .call()
    .content();
```

THE MAGIC OF THE ABSTRACTION

You never write any if/else or switch to decide when to use the tool. You simply say "here is a tool, use it if you need it". Ask for a joke and the model ignores the tool. Ask for the current time and the model decides to call it. **Spring AI executes the tool**, takes the tool's raw result, blends it with the model's own response, and returns the combined answer to the client. The developer only arranges the tool; the model decides when to call it.

Tools can take parameters and you can have many

To get the time in a specific city, accept a parameter. The model extracts the argument ("Boston") from the user's question and passes it in:

```
@Tool(description = "Get the current date and time for a given time zone")
public String getCurrentDateTime(String timeZone) {
    return ZonedDateTime.now(ZoneId.of(timeZone)).toString();
}
```

You can register any number of tools. In the class a second tool called a live news API (newsapi.org) using `RestTemplate` . With both a date/time tool and a news tool present, the model correctly picks the news tool for "did RCB win the IPL in 2026?" and answers with fresh data, resolving the earlier hallucination. This is how tool calling defeats the knowledge cutoff.

FLOW, IN ONE LINE

Client request → your app calls the model with the prompt plus available tools → the model reads the tool descriptions and decides to call one → Spring AI runs the tool → the tool result goes back to the model → the model merges it into a final answer → the client receives it. (MCP, a related concept, is covered later in the course.)

11 Key takeaways

- Spring AI adds AI capabilities to Spring Boot apps with a clean, provider agnostic abstraction.
- Configure the key in `application.properties` (`spring.ai.openai.api-key`).
- Use **ChatModel** for a quick provider specific call, or **ChatClient** for a higher level, provider agnostic one.
- Models are stateless by default. Add **advisors** (chat memory + a conversation id) to make the app remember.
- Models have a **knowledge cutoff**. Use **tool calling** (and later RAG) to bring in current or private data.
- With `@Tool` , you just offer tools; the model decides when to use them, and Spring AI runs them for you.
- The ecosystem changes fast, so understand the concepts rather than memorising exact APIs.

COMING NEXT

Why the conversation id is now required (explained fully next session), plus how RAG, tool calling, and MCP differ and work together, and structured output. This was only class one of Spring AI.